

UConn Quant: Trading Algorithm

School of Computing, University of Connecticut

Andrew Khan, Connor Jackson, Peter Petropoulos, Preston Rainey, Sagar Patel

Professor Curry

CSE 4939W

12/05/2025

Introduction – Sagar Patel:

Systematic trading strategies and algorithms are built to transform historical price and market data into objective, rule-based investment decisions. For our project, we developed a trading algorithm that aims to combine momentum strategies with common risk-management techniques to create a disciplined approach to investments that would simulate certain hedge fund approaches. Instead of relying on opinion and personal judgement, the algorithm is built to apply a consistent framework that evaluates market conditions in the same way no matter what the public opinion is. This creates a reliable and repeatable process with clear logic that can be tested, validated, and refined through analysis and back testing.

Our algorithm integrates three major components: a long-term momentum signal for asset selection, a trend filter for market condition analysis, and built in risk controls that limit our exposure. Momentum determines which assets and securities have exhibited strength and potential in the target timeframe, contributing to our asset allocation. The trend filter helps evaluate whether macro conditions support taking risk at all. The risk controls consist of volatility targeting, leverage limits, and drawdown controls that ensure the overall portfolio remains within a stable range.

This algorithm design reflects a common philosophy in quantitative finance. This is the idea that returns are generated by the signal, but viability is determined by risk management. Momentum strategies have historically produced strong performance in the long term, but they also tend to be notorious for facing sharp drawdowns and having a sensitivity to market regimes. There is also the issue of volatility in the financial markets having a tendency to cluster. This makes consistent risk exposure difficult without highlighting specific risk scaling mechanisms. Our algorithm attempts to address these concerns by incorporating trend filtering, volatility targeting,

and drawdown constraints. The intention is that this will result in a more resilient momentum strategy that maintains strong return potential while also reducing exposure in bear markets and economic downturns.

Overall, the trading algorithm we have implemented is built of components that each have a distinct and refined role. Since trading rules are explicitly outlined in the algorithm, the model and underlying strategy can be evaluated across different timelines and environments through back tests on historical data. The configurations can then be adjusted and retested in order to refine the performance of the strategy in the long term over various market conditions.

Related Work – Sagar Patel:

Momentum has historically been one of the most consistently documented anomalies in financial markets and formed the foundation for the trading algorithm that we developed as a team.

Momentum was formally established by Narasimhan Jegadeesh and Sheridan Titman in 1993 when they demonstrated that stocks with strong returns over the prior 3-12 months continues to outperform other similar stocks in following periods. Their findings challenged the Efficient Market Hypothesis because they showed that past price performance contained some predictive power. We took this idea of momentum and implemented a refined strategy, which is called the “12-1 momentum strategy.” This approach uses the past twelve months of returns and excluded the most recent month to avoid short term mean reversion. This version of momentum has been shown to produce higher risk adjusted returns by reducing sensitivity to short term price reversals (Jegadeesh & Titman, 2001).

The uses of momentum strategies do not simply end at U.S. equities. The phenomenon has been shown to be effective for other asset classes including international equities, commodities, fixed

income securities, and even currencies. In 2013, Clifford Asness, Tobias Moskowitz, and Lasse Pedersen, demonstrated the global presence of momentum in nearly every major asset category. These findings strengthened the idea that momentum emerged from psychological, behavioral, and structural market features as opposed to simple inefficiencies. A key contributor to momentum could be the psychological tendency of investors to underreact to new information, leading to slow movement of price that can further be taken advantage of.

The institutional relevance of momentum has also been explored and found effective. In 1997, Mark Carhart incorporated momentum into his four-factor model and showed that it improved the explanatory power of traditional asset pricing models. This helped establish momentum as a key component of modern investing, leading to it being widely adopted by hedge funds, quantitative trading firms, and other institutional investors. Hedge funds in particular have embraced momentum because of the diversification benefits it provides along with its strong historical performance during times of market volatility. For example, trend following, which relies heavily on momentum signals, has shown the ability to perform well in both bear and bull markets, further contributing to the popularity of momentum strategies in the industry.

Despite the many benefits of momentum strategies, there are also risks that need to be addressed. Momentum crashes, which are sharp reversals that follow periods of strong performance, pose a significant risk and outline a key vulnerability of momentum strategies. To address this issue, risk management tools such as volatility scaling, trend filters, and drawdown based deleveraging are utilized to stabilize the strategy in risky environments. These techniques reduce exposure during volatile markets and, as a result, have been incorporated into our algorithm.

Overall, our research has supported the empirical strength of the 12-1 momentum strategy while highlighting the necessity for risk management controls. Furthermore, the widespread use of

momentum strategies by hedge funds and other systematic trading firms continues to validate their relevance in modern investments. All in all, our findings have deepened our knowledge of both the strengths and weaknesses of momentum trading strategies. This has directly informed many design choices in our model, including those to combine traditional momentum strategies with modern risk management techniques to create a more stable and resilient trading algorithm.

Configurations – Sagar Patel:

In our algorithm, we have a plethora of configurations broken into two main sections: core strategy configurations and risk management configurations. The first of the core configurations is rebalance frequency (REBAL_FREQ). This variable describes how frequently we rebalance our portfolio based on our strategy. Due to the traits of the 12-1 momentum strategy we are implementing, we decided to rebalance on a monthly basis. The next configuration we have is the “TOP_PCT_PER_SECTOR.” This determines what percentage of the top of each sector of the S&P 500 we take long positions on. In order to ensure we capture truly high performing securities and minimize our chances of regression, we decided to set this at 5%. The configuration called “MAX_NAME_WEIGHT” sets a cap for the weight of each stock in the overall portfolio. In order to minimize the impacts of one singular security on our overall returns, we set this to 5%. Next, we have “COST_PER_TURNOVER,” which sets a cost of 10 basis points per dollar traded to simulate the costs of trading in the real world. We also have “VOL_WINDOW,” which determines the lookback window for volatility. We set this at 20 to look at about one month of prior data for our volatility calculations. “USE_TRI_FIRST” essentially determines if we simply use the price index or if we utilize the total return including dividends. Since dividends help describe the overall return and performance of a security, we set this configuration to “True.”

After the core configurations, we have our risk management configurations that we can further adjust to refine the performance of our algorithm. Since we have a momentum strategy, we would ideally want to trade when the market is performing very well. To ensure this, we have the “USE_TREND_FILTER” configuration, which enables a rule that only allows the strategy to take positions when the overall market is in an uptrend. This helps avoid trading during major bear markets, which will reduce our drawdowns. The “TREND_MA_WINDOW” helps compute this market trend. It is the number of days used to compute the moving average that defines the market trend. A longer window makes this filter slower but more reliable. We decided to use 200 days as our value as this would minimize false signals without adversely affecting our strategy. The “TREND_TICKER” variable checks market trend conditions based on an ETF, trading only if the asset is above its moving average trend line. We chose the SPY, which tracks the performance of the S&P 500, providing us with a good estimate of market conditions.

Since we have a trading algorithm that would ideally be used in a professional setting, we need to balance our returns and volatilities. This is why we have the USE_VOL_TARGETING configuration, which enables volatility targeting. This will mean that position sizes will be dynamically adjusted to maintain a stable portfolio volatility, smoothing returns and preventing overexposure in turbulent markets. After enabling volatility targeting, we set our target volatility to 10% to provide some flexibility for the algorithm to work while minimizing the potential losses. “VOL_LOOKBACK” defines how many past days are used to estimate realized volatility for position sizing. Shorter windows make volatility estimates more responsive, but slightly noisier. In order to balance this, we landed on a lookback of 30 days to minimize noise while still having strong results. Next, we have our leverage boundaries. “MAX_LEVERAGE” determines the maximum exposure after volatility scaling. This prevents the strategy from taking overly

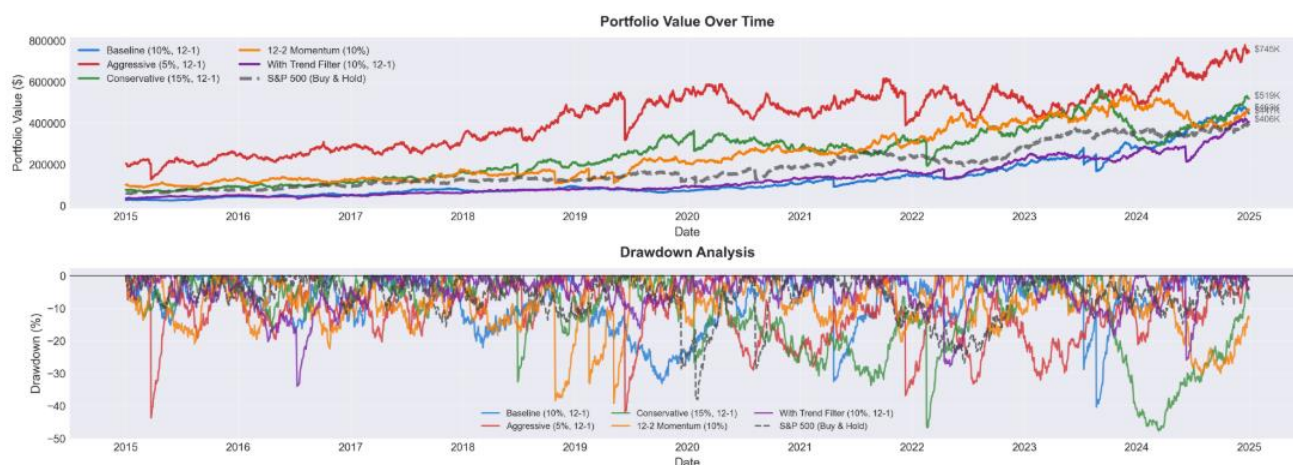
large risky positions. We set this value to 1, which means that we can take a position up to our entire portfolio value, but we will not utilize leverage. We set our minimum leverage to 0.25 to ensure that we are participating in the market to some degree at any given point.

We also have drawdown controls that reduce exposure if our strategy experiences large losses. This control helps protect capital during prolonged losing periods. This allows us to deleverage our portfolio and protect ourselves in the long term. “DD_THRESHOLD_1” is set to 10%. This means that when our strategy is down 10% from its peak, exposure is cut to 75%. This softens further losses while keeping some upside exposure in the event of a recovery. The second drawdown threshold of 15% cuts our exposure to 50% when we have a 15% total drawdown. This further protects our capital as losses deepen. Finally, we have a third drawdown threshold of 25%, which is when positions will be fully exited in an attempt to stop catastrophic losses in a heavy bear market.

All in all, these configurations allow us to fine-tune our portfolio performance, balancing returns and risk. As a professional trading algorithm, we should be able to create strong positive returns that are sustainable in the long term. By adjusting these configurations, we have been able to improve our results over our test period from 2015-2020. We have improved our returns by over 50% from 9.58% to 14.49% while keeping a relatively steady volatility of around 13%. We were also able to increase our risk adjusted return, measured by a Sharpe Ratio increase from 0.75 to 1.07. Our max drawdown also improved from -25.45% to -21.81%. These results overall demonstrate the importance of these configurations in the results of the strategy as a whole. Going forward, we will continue to fine tune these configurations in an attempt to maximize the long term performance of our trading algorithm.

Preliminary Results - Andrew Khan:

Our back tests show that the strategy can grow capital faster than a simple S&P 500 buy and hold while keeping risk at a controlled level. Using a 12-1 momentum signal on S&P 500 stocks with our risk rules, the baseline configuration reached about 447 thousand dollars from the starting value, with a compound annual growth rate near 15.6 percent. Over the same period the S&P 500 grew to about 388 thousand dollars with a 14.0 percent annual rate. In plain terms, the strategy earned more each year than the market even before we tuned the parameters.

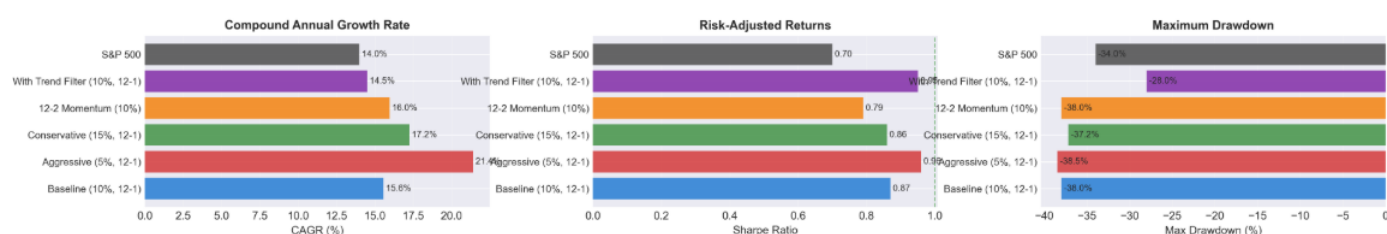


Risk adjusted results tell a similar story. The baseline Sharpe ratio was about 0.87 compared with 0.70 for the S&P 500. That means each unit of risk produced more return than the index.

Maximum drawdown for the baseline stayed around negative 38 percent which is similar to major equity drawdowns in the sample. So the strategy did not avoid all painful periods, but it paid more for each unit of volatility it took. The daily return distribution plots support this point. Most days cluster close to zero with a slight positive tilt, and the tails look similar to the benchmark. The edge comes from a small positive shift in many days rather than a few extreme wins.

Changing position sizing rules gave a clear tradeoff between growth and risk. The aggressive configuration that concentrated in the top 5 percent of names per sector produced the strongest growth with a CAGR near 21.4 percent and a final value around 745 thousand dollars. Its Sharpe ratio rose to about 0.96 which means the extra return was not just noise. Drawdowns were still large, near negative 38 percent, so the aggressive version suits a risk tolerant investor. The conservative version that capped single name risk more tightly landed near 17.2 percent CAGR and 519 thousand dollars with a Sharpe around 0.86. This confirms that narrowing the portfolio to the strongest names can raise returns, but it keeps you exposed to the same equity style risk.

The configuration tests on TOP_PCT reinforce this point. When we only held the top 5 percent of stocks in each sector the strategy delivered the highest return and the best Sharpe ratio. As we widened the basket to 8, 10, and 15 percent, returns and Sharpe both fell while volatility and drawdown drifted higher. In other words, dilution hurt more than it helped. Spreading into weaker momentum names did not provide much real diversification and it cut the edge of the signal. The risk versus return scatter plot shows this visually. The 5 percent setting sits in the upper left region with higher CAGR for a given volatility level while the wider baskets move down and to the right.



We also compared two momentum lookback rules. The standard 12-1 specification, which skips only the most recent month, slightly outperformed a 12-2 version that skips the last two months. The 12-1 rule posted about 17.4 percent CAGR versus 16.0 percent for 12-2. Sharpe ratios were similar near 0.8, and both versions reached the same worst drawdown of about negative 38 percent with similar volatility near 22 percent. This suggests that in our sample the extra month of data that 12-1 uses still contains useful trend information and does not introduce harmful mean reversion. Rolling one year return plots back this up since the 12-1 line generally sits slightly above 12-2 without showing extra instability.

The trend filter configuration shows the role of risk management. Adding a 200 day moving average filter on the market reduced drawdowns from roughly negative 38 percent to around negative 28 percent while keeping CAGR near 14.5 percent and Sharpe near 0.95. That means the strategy gave up a small amount of return in exchange for materially smaller peak to trough losses. For many allocators this kind of smoother ride is worth the cost. The drawdown chart shows that filtered equity curves avoid the deepest troughs during stress periods and recover more quickly once the market turns.

We also tested stability across different time windows. Over the full 15 year period from 2010 to 2024 the strategy produced a CAGR near 21.1 percent with a Sharpe ratio around 1.12. Over the last 10 years it earned about 17.3 percent with a Sharpe near 0.95. Over the last 5 years it earned about 17.4 percent with a Sharpe near 0.85. The exact numbers move with the sample, but the pattern is consistent. The strategy stays profitable, keeps Sharpe above the S&P 500, and experiences similar worst drawdowns near negative 38 percent. Rolling Sharpe plots show the same behavior. There are stretches where the ratio dips toward zero during tough markets, but it spends much of the sample in positive territory, often above one.

Strategy	CAGR	Sharpe	Max DD	Vol
5-Year (2020-2024)	17.43%	0.85	-38.0%	22.2%
10-Year (2015-2024)	17.26%	0.95	-38.0%	18.8%
15-Year (2010-2024)	21.11%	1.12	-38.0%	18.8%

Taken together, the results support three main points. First, the 12-1 momentum signal on S&P 500 stocks has been a strong source of return in our sample. Second, simple but explicit risk tools such as trend filters, volatility targeting, and position caps can shape that raw edge into profiles that match different risk preferences from conservative to aggressive. Third, the strategy behaves in a stable way across parameter changes and time windows which gives us confidence that it is more than overfitting and that the signal is robust.

At the same time there is clear room to improve. Drawdowns remain large in equity crashes even with filters. The strategy only trades one asset class and uses price based signals. Future work with broader universes, alternative data, and regime models along with a full cloud deployment should focus on two goals. Keep the return profile we see here and reduce the depth and length of the worst drawdowns.

The longer we studied the results, the more clear it became that the strength of the strategy rests on a simple pattern. Markets reward persistence in price movement. When an asset shows steady strength over many months it often keeps rising. This is not a guarantee but the data shows that the pattern is strong enough to form the base of a systematic process. The steady outperformance

of the baseline version shows how a basic rule can still deliver a meaningful edge when paired with consistent execution. The fact that the strategy beat the benchmark before we added extra rules supports this idea. It also shows that the improvements in the more advanced configurations are built on a real foundation rather than noise.

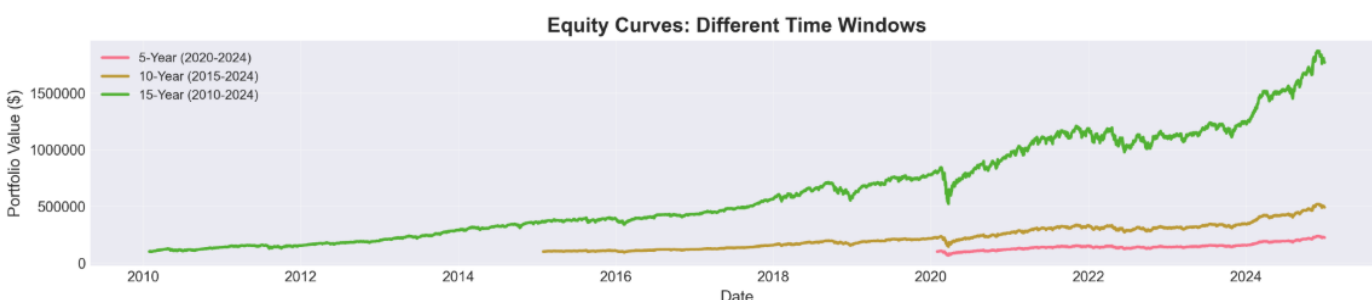
Another point that stands out is how sensitive the system is to small changes in breadth and concentration. When the portfolio holds only the strongest names it captures clearer trends and avoids names that drift sideways. This is why the narrow baskets did better in every test. A wider basket adds exposure to weaker names that slow the portfolio down. The impact is not always obvious in a single year, but across long periods the effect compounds. A few percent lost each year from dilution adds up to a sizable gap in final value. This helps explain why many professional momentum strategies lean toward concentration even when it increases drawdown risk. They accept that the path will be rough at times but they want to keep the core edge intact.

The drawdown behavior is another important part of the results. The strategy earns high returns but still falls sharply during market stress. That is not surprising because the system is long only and holds stocks that have been strong. These names often fall just as hard as everything else during crises. The trend filter helps by cutting exposure when the market weakens, but it cannot remove all losses because it reacts with a delay. This is the nature of trend following. You benefit from the long upward moves but you give back some gains when the market turns. The test results show that the filter improves the ride without breaking the strategy. It cuts the worst losses but keeps most of the return. This makes the approach more practical for real world use where investors care about both sides of the equation.

We saw similar behavior in the volatility of returns. The portfolio has a positive tilt on most days, but it does not rely on rare extreme gains. This is a healthier pattern than strategies that

depend on a few huge outliers. It means the system can scale because the edge comes from many small favorable outcomes. It also means the system is less likely to be overfit because the return comes from broad behavior rather than isolated periods. A strategy that relies on a few clusters of lucky trades is fragile. A strategy that earns a little bit across many months is more stable. This was clear in the rolling return charts where the line stayed above the benchmark for long stretches.

The stability across time windows matters as well. Many trading systems look strong in one decade and weak in the next. This is a sign that the underlying pattern might be tied to a specific environment. Our results show steady performance across the 15 year, 10 year, and 5 year windows even though those periods include very different market conditions. The period from 2010 to 2024 includes a long bull market, a sudden pandemic crash, a rapid rebound, and rising interest rates. A strategy that works across all of these conditions is probably capturing a real feature of markets. It does not mean it will always work, but it does mean the signal is not limited to one narrow environment.



A key lesson from the tests is that the raw momentum signal is not enough on its own. The strategy needs risk controls to shape the path of returns. Position caps prevent single stocks from dominating the portfolio. Volatility targeting can help smooth the ride by adjusting exposure during turbulent months. Trend filters reduce drawdowns when the entire market weakens. These

tools do not create returns but they protect them. They make the strategy easier to hold through tough periods which is important for any real user. A system that earns high returns but experiences repeated deep drawdowns is hard to follow in practice. Investors often abandon such systems at the worst times. A more controlled path is easier to stick with.

We also came away with a clearer sense of what limits the strategy. The worst drawdowns are still large which means any live version will need better protection. Moving into more asset classes may help. Global equities, bonds, commodities, or factors outside price could soften the blow because they move differently from the S&P 500. Adding alternative data might help the system adapt faster to changing conditions. Regime models could help the system change behavior when volatility rises or macro conditions shift. None of these improvements guarantee smoother results, but they provide more tools to control risk without altering the core signal.

In the end, the results show that a long only momentum strategy on large cap equities can produce strong returns with clear and measurable behavior. It is not perfect. It is not smooth. But it is consistent across many tests and that makes it valuable. The next phase of work will aim to keep that consistency while reducing the worst periods. If we can keep the long term profile while improving the depth of losses, the system becomes more practical for real use and more reliable for future research.

Backend: Materials and Methods - Andrew Khan

The algorithmic backend is composed of four primary subsystems: a multi-source data ingestion pipeline, a portfolio execution engine, an automated scheduling system, and a cloud storage layer built on AWS S3. Together, these components form the infrastructure that transforms raw market data into live brokerage orders without requiring manual intervention. This section focuses specifically on how those components were implemented and how they connect to the AWS environment.

Data Ingestion Pipeline - Andrew Khan

The data layer is built around a unified interface in `'data_pipeline.py'`, which abstracts across four distinct external sources and consolidates them into a single consistent format at runtime. The primary source for historical price data is a Bloomberg Terminal export stored as Apache Parquet files, partitioned by year under `'data/prices_parquet/'`. Parquet was chosen over CSV because its columnar storage format enables substantially faster reads when filtering by date range or ticker, a relevant performance consideration when the pipeline loads millions of rows spanning thirty years of price history. Each record contains a daily close price (`'px_last'`), a total return index that compounds dividends (`'tri_gross'`), and trading volume for every S&P 500 constituent ever included in the index.

Because Bloomberg data has a fixed cutoff date, the pipeline extends coverage to the present day using a Yahoo Finance top-up module (`'price_topup.py'`). At the start of each run, this module identifies the last date present in the Bloomberg dataset and fetches all subsequent daily bars through the `'yfinance'` library. The result is stored as `'data/cache/price_topup.parquet'` and

merged with the Bloomberg dataset in memory at runtime, producing a seamlessly continuous price history with no gap between the historical and live data.

Macro indicators, the CBOE Volatility Index, the 10-year/2-year Treasury yield spread, and the BAA corporate credit spread, are retrieved from the Federal Reserve Economic Data (FRED) API via the `'fredapi'` library. A fresh pull is triggered at the start of every rebalance to ensure that the regime classifier is operating on current readings rather than stale cached data. The output is saved to `'data/cache/macro_data.csv'`. Gold prices for the GLD ETF sleeve are handled through a parallel module, `'gold_fetcher.py'`, which queries Yahoo Finance and caches the result in `'data/cache/gld_prices.csv'`. A third alternative, `'alpaca_fetcher.py'`, provides a fallback price source by querying Alpaca's market data API directly in cases where the Bloomberg export is unavailable. This fallback makes the backend portable to environments without a Terminal license.

AWS S3 Data Layer – Andrew Khan/Preston

In the deployed version of the system, the `'data/cache/'` directory corresponds directly to an AWS S3 bucket. The Parquet price files, macro indicator cache, and GLD cache are all written to and read from S3 paths that mirror the local directory structure. S3 was chosen because it provides durable, versioned object storage that is accessible to compute services without requiring a persistent server, and because Parquet files stored in S3 can be queried efficiently by downstream services including Pandas and AWS Athena. Columnar reads mean that only the date slices and tickers actually needed for a given rebalance are loaded into memory, which significantly reduces both runtime and cost when the dataset grows.

The constituent membership file (`constituents_long.csv`) and sector mappings (`sectors.csv`) are also stored in S3 as static reference assets. Because these files rarely change, they are versioned rather than refreshed on every run, which reduces unnecessary API calls and makes the dataset reproducible from any compute environment.

AWS Lambda Layers and Deployment - Andrew Khan/Preston

Lambda Layers in AWS are a packaging mechanism that separates a Lambda function's application code from its external dependencies. The algorithmic backend depends on a non-trivial set of Python libraries including pandas, numpy, scipy, alpaca-py, fredapi, and PyArrow. Bundling these into a Lambda Layer rather than including them inline with each function keeps the deployment package small and enables the same dependency set to be shared across multiple functions without redundant uploads. The `requirements.txt` at the project root defines the exact set of packages that are compiled into these layers. When a dependency version needs to be updated, only the layer is rebuilt, the function code itself does not need to be redeployed, which simplifies version management.

The lightweight, time-sensitive steps of the pipeline, primarily the FRED data refresh and GLD price fetch, are well-suited to Lambda functions because they execute in seconds and run infrequently. Triggering these steps through Lambda rather than a persistent process eliminates the cost of keeping a server running between rebalances, since the algorithm only needs to execute once per month during live trading. The heavier compute steps, such as loading the full Parquet dataset and running signal generation across all constituents, exceed Lambda's execution and memory constraints and are intended for EC2 in the production architecture.

Execution Engine - Andrew Khan/Preston

The execution layer consists of three modules that work in sequence. ``alpaca_broker.py`` wraps the Alpaca REST API and handles authentication, account queries, position lookups, and order placement. API credentials are loaded from environment variables or a local ``.env`` file and are never hardcoded, which prevents accidental exposure through version control. A single boolean flag in ``config/settings.yaml`` switches between Alpaca's paper trading endpoint and the live endpoint, meaning the same codebase handles both simulation and production without any code changes.

``rebalancer.py`` translates target portfolio weights into a concrete list of orders. Given a dictionary mapping each ticker to a target weight and the current account value, the rebalancer computes the target dollar allocation for each position, retrieves the current holdings from the broker, and generates orders for any position where the difference between target and current exceeds ten dollars. That threshold prevents the system from generating orders for trivial rounding discrepancies. Bloomberg-format ticker symbols are automatically cleaned to the plain format Alpaca expects before any order is submitted.

The execution sequence is ordered deliberately to prevent capital shortfalls. All open orders are cancelled first to clear any stale orders from prior sessions. Sell orders are submitted before buy orders so that proceeds from liquidated positions become available before new purchases are placed. A two-second pause separates the two phases to allow sell fills to begin processing.

Within each phase, a 0.5-second delay is enforced between individual submissions to stay within Alpaca's API rate limit of 200 requests per minute.

Scheduling and Automation - Andrew Khan/Preston

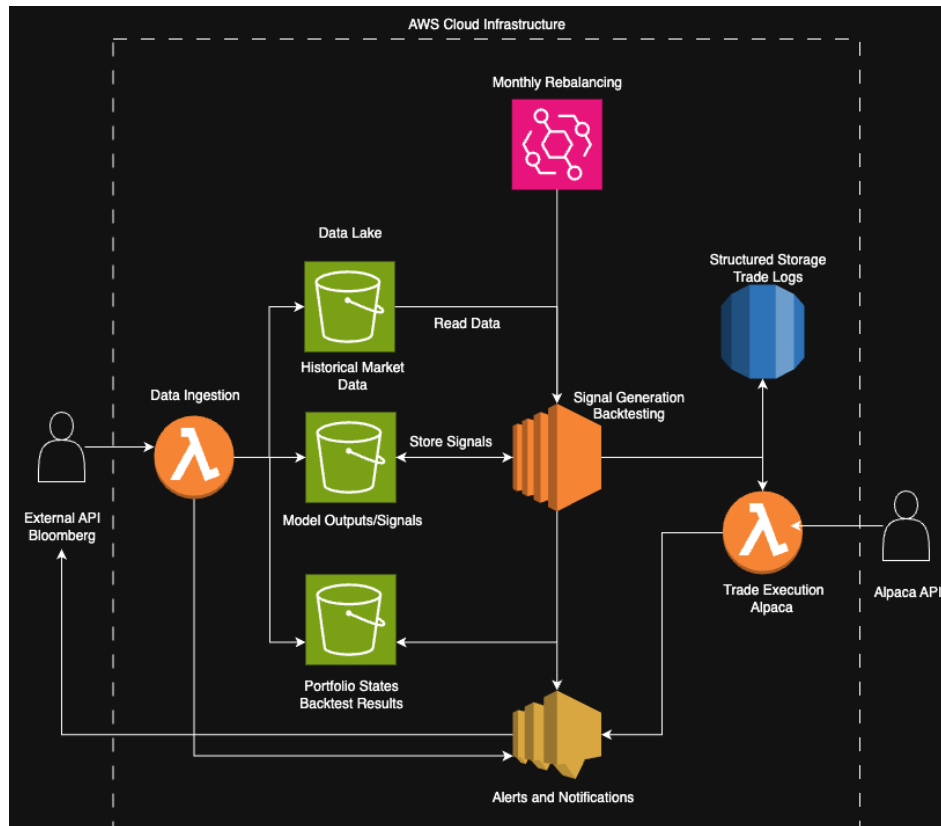
`scheduler.py` wraps the full rebalance workflow, data refresh, signal computation, portfolio construction, risk controls, and trade execution, into a single callable function. In persistent mode, this function is triggered by APScheduler's `CronTrigger`, configured to fire on the first business day of each calendar month at 10:00 AM Eastern Time. In one-shot mode, `run\_at\_open.py` queues a single rebalance for the following morning at 9:30 AM without requiring a persistent background process. This design allows the scheduler to operate either as a long-running process on an EC2 instance or as a function triggered by AWS EventBridge on a cron expression, depending on the deployment environment, without any modification to the underlying rebalance logic.

Conclusions & Future Work – Preston Rainey:

This project developed a fully operational momentum based long only trading algorithm supported by a complete end to end data pipeline. As a long only strategy, the overarching objective of the system is to outperform the S&P 500 on a risk adjusted basis, reflecting the same benchmark driven performance standards used throughout the quantitative asset management industry. The system is capable of pulling historical and daily market data, ingesting the inputs, generating trading signals, constructing portfolios, and running reproducible back tests. By building these components into a complete pipeline, the team ensured that data flows seamlessly from acquisition to analysis, enabling consistent evaluation of the strategy across long historical periods and reducing opportunities for human error or confusion. This infrastructure not only validates the algorithm's performance, but also establishes a foundation for continuous research and production level deployment. The primary objective of the project was to transform a simple factor based momentum algorithm into a systematic, transparent, and testable investment framework. Through iterative development, the system now incorporates standardized

preprocessing routines, signal generation, and a documented rebalancing procedure capable of generating stable and interpretable results. The back testing portion of the algorithm provides clear historical performance metrics and allows for rapid experimentation with weighting rules, risk constraints, and other configurations. Beyond the algorithm itself, significant effort was dedicated to designing an engineering architecture that could scale beyond a local development environment. The systems for data ingestion, signal generation, and portfolio output allow the system to operate reliably and make future cloud integration straightforward. These design choices demonstrate how a prototype can be transformed into a production ready quantitative pipeline. Overall, the completed project delivers a validated training model, a functional data infrastructure, and a scalable software foundation. The steps that were taken so far position the system for the next stage of development, including cloud deployment, alternative data ingestion, broader market universes, and machine learning enhancements which are outlined in the subsequent sections.

A significant direction for future development involves deploying the trading algorithm onto a scalable cloud infrastructure using Amazon Web Services (AWS). While the current system runs in a local environment, migrating to AWS would enable automated data ingestion, continuous operation, reproducible back testing, and improved reliability. The figure below outlines the proposed architecture, which integrates several AWS managed services to create a fully automated pipeline for signal generation, portfolio construction, and performance monitoring.



In the proposed design, Amazon Simple Storage Service (S3) would function as the primary data lake for all market data, model outputs, and portfolio states. Historical prices, factor signals, and daily sector allocations would be stored in various S3 buckets. Using S3 allows the backend team to maintain a unified, immutable dataset that is accessible to compute services without risk of local corruption or inconsistency. Additionally, storing back test outputs such as daily returns in S3 enables reproducibility and facilitates ongoing model validation. To support continuous operation, daily price updates and economic indicators must be fetched and pre-processed without manual intervention. AWS Lambda provides an ideal mechanism for this task: a Lambda function can be triggered once per trading day or when needed to query external APIs (ex: Bloomberg, AlphaVantage, Alpaca), clean the incoming data, and write the results to S3. Because Lambda is event-driven and serverless, the system incurs cost only when executed, making it efficient for relatively lightweight ingestion workloads. Full signal generation, multi-

factor ranking, and long-horizon back testing require computational resources beyond the scope of Lambda. As future work, these workloads will be migrated to Amazon EC2 instances configured with Python specific libraries and optimized storage access. EC2 allows on-demand scaling: small instances may be used for daily production runs, while larger compute nodes can be provisioned temporarily for research back tests. This separation between data ingestion and heavy analytics provides layers of security and reliability. Although S3 is effective for storing raw and large-scale data, the system also requires structured storage for trade logs, realized performance, and metadata. Amazon Relational Database Service (RDS) will be used to store executed allocations, timestamped rebalancing decisions, and aggregated performance metrics. Using a relational scheme facilitates downstream analysis, enables efficient querying, and provides a consistent source of truth for auditability. The algorithm employs a monthly rebalancing cycle for its momentum and cross sectional allocation framework. AWS Event Bridge (or CloudWatch) will be used to trigger the necessary compute pipeline on the first trading day of each month. Event Bridge will orchestrate the workflow by invoking the EC2-based signal code, which accesses updated data in S3, recalculates rankings, and writes new portfolio weights to both S3 and RDS. As a final component, Amazon Simple Notification Service (SNS) will provide real-time monitoring and operational alerts. For example, SNS can deliver notifications via email if data ingestion fails, if a rebalancing job does not complete, or if performance deviates slightly from expectations. Incorporating SNS improves system transparency and ensures that the backend team can intervene promptly when necessary. Deploying the algorithm on AWS will transform the current prototype into a production ready trading algorithm. The cloud-based system will allow automated daily processing, minimize computational bottlenecks, and preserve a detailed record of model outputs and performance.

Lastly, modularizing the system provides a scalable framework that can support potential future additional features, such as intraday signals, or alternative data ingestion.

In addition to deploying the system on AWS, several methodological improvements could meaningfully improve the algorithm's performance, robustness, and real-world applicability. These enhancements reflect common practices used by quantitative hedge funds that evolve from simple factor models into diversified, multi-regime, institutional grade systems. The following outline several directions for future work and the expected impact of each improvement. A natural expansion of the current S&P 500 focused strategy is to broaden the tradable universe to include additional equity indices and global markets. Incorporating an index like the Russell 2000 would introduce small-cap exposure, a segment historically associated with a persistent size premium. Preliminary academic research suggests that diversification across size tiers and international markets can add approximately 1-2% compound annual growth rate (CAGR) over long horizons, primarily through exposure to independent sources of return. Expanding the universe also increases the statistical robustness of the momentum factor signals by providing a larger cross-section of securities. As the system becomes older, integrating alternative data can enhance signal quality and reduce model lag. One type of alternative data that could improve returns is news and sentiment data. Natural language processing based sentiment scores derived from news feeds, social media, and company transcripts can help identify inflection points earlier than price data alone. In addition, analyst estimate revisions and earnings surprise trends have been shown to complement traditional price momentum. These sources introduce forward-looking information and reduce the model's reliance on backward looking price history. Studies indicate that combining price momentum with alternative data can yield an additional 2-3% CAGR while lowering drawdown. Future iterations of the system may incorporate machine-

learning models to capture nonlinear relationships that traditional frameworks cannot fully represent. A gradient boosting machine could be used to rank stocks or sectors based on many interacting signals. In addition, a Markov model could be used to classify markets into volatility regimes (ex: trending, choppy, high-variance), allowing the model to adapt signal weights or risk levels dynamically. Machine-learning enhancements can improve Sharpe ratio by adjusting to market environments where classical linear momentum fails. This is especially relevant during high-volatility periods when regime specific behavior dominates. Professional systematic funds also frequently incorporate volatility signals, most notably the VIX to modify exposure based on market stress. For example, the strategy could reduce gross exposure when VIX rises above a predefined threshold to avoid sharp trend reversals. Such mechanisms can meaningfully reduce drawdowns and stabilize returns particularly during periods of rapid momentum crashes. Integrating volatility overlays is expected to improve risk adjusted returns even if raw CAGR remains consistent.

Related Work: - Connor

Based on the current backend developed by several other members of the team, this section delves into the related work that informed the design and implementation of our latest version of the frontend system. Through examination of existing platforms, current UI/UX exercises, and known data visualization methods, our architectural decisions and development choices will be contextualized for the following Materials and Methods section.

Financial dashboards are a key resource for almost every modern investment platform. Without them, users would not be able to do things like check their portfolio, look at market conditions, and track performance on specific assets. When we first started to develop our project, we noticed that the most popular trading applications combine resources like charts, news, and

analytics into one, user-friendly screen. These resources have become a staple in many applications and have set the precedent for what an investing interface should look like. Viewing these platforms helped shape the resources and design we wanted for our own frontend.

A platform that had a major influence on our design was Robinhood, which is historically known for its simple yet informative interface. Robinhood maintains focus on an uncluttered screen with large visual elements to make complex market data easier to understand. The main chart is the center of the website, and color cues (green for positive, red for negative) simplify viewing trends for users. The minimalistic design of the site has made it incredibly popular for first-time investors and displays how the prioritization of ease of use is vital for an investment dashboard.

On the other end of the spectrum, platforms like Charles Schwab have dashboards that are a little more complex. These types of platforms are for more experienced investors who want more information like performance summaries, sector breakdowns, and customizable chart tools. They have more “traditional” dashboards and charts, which are useful for more advanced trading behavior and analysis.

Looking at both types of platforms, there is an obvious contrast in the minimalist and data-heavy dashboards; we wanted our dashboard to encapsulate components of both. We aimed to create a layout that was approachable and not daunting, while also providing users with a plethora of information to grasp their financial status.

When considering the design of a financial dashboard, the amount of information displayed on screen needs to be balanced with the clarity of its meaning. There are many Human-Computer Interactions (HCI) studies that emphasize how users can become overwhelmed if the UI is too complex. These studies have also shown that users can interpret data most effectively when there

is a logical grouping of data and a clear visual hierarchy. Maintaining a reduction in cognitive load, even by doing small things like keeping the spacing consistent and removing unnecessary elements, can help a user make quicker and more confident decisions. We kept these ideas in mind when designing our layout, trying to make it as clean as possible while prioritizing the basic and important market data, rather than jamming it with every possible piece of information.

Along with our psychological consideration with HCI studies, we also examined reputable UI/UX research. The Nielsen Norman Group, a consulting and research firm that specializes in helping companies design friendly UX products for users, had findings that supported our earlier beliefs. One of the main “usability heuristics for user interface design” is “consistency and standards,” centering on a belief that “failing to maintain consistency” on our site compared to other “may increase the users' cognitive load by forcing them to learn something new” (). Users rely on consistency to generate an understanding of how an interface works. Elements in high-attention areas, such as the center of the screen, are processed quicker by a user, resulting in a clearer understanding. These principles shaped our design, causing us to structure our dashboard with vital information in prominent locations, with less important elements in visually weaker areas.

With our design in mind, we needed to consider the way we were going to code and execute our idea. Although the dashboards of many financial applications utilize frameworks such as ReactJS, we decided to develop our frontend using regular HTML, CSS, and JavaScript. This decision may seem confusing but based on the scope of our application and the deployment considerations being made, it makes perfect sense for our group. Our design layout is straightforward and does not have much client-side routing, so developing using a whole frontend framework would introduce useless complexity. Plain HTML and JavaScript give us

full control of the UI without worries of handling additional build tools and framework-specific protocols.

Along with removing unnecessary challenges, deployment simplicity was a major factor in our decision. Projects built using frameworks generally rely on package managers and building processes that can cause complications with hosting. Using plain HTML, CSS, and JavaScript allows for a simple deployment of our static site with minimal configurations. This will also make it easier for anyone from our team to update or troubleshoot the website, regardless of whether they have experience with a specific framework or not.

We were able to migrate our site from a local Flask application to a cloud-based service by using Amazon Web Services (AWS). Our site was originally a static website that was locally hosted using Flask, but we were able to migrate this design to be hosted on an S3 bucket, a cheap and reliable service offered by AWS for static hosting. This change made a substantial difference, as it eliminated the need for a Flask server, Jinja2 templates, and local routes as a whole. Files referenced that previously needed Flask's `url_for()` helper are now plain relative paths, and the "defer" attribute is now used in the JavaScript to make sure the DOM is fully parsed before execution. Moving the site from Flask to AWS has made it significantly easier to maintain it, as instead of needing to run the Flask app every time we want to see the site, the S3 bucket remains running across AWS sessions.

Along with the change in deployment method, we also newly considered the modern relevance of AI assistance in finance as a part of our research. Many financial platforms, such as Bloomberg Terminal, have integrated the use of AI chat assistants to assist inexperienced users in learning and making smart financial decisions. The platforms that have these AI bots influenced our decision to include a dedicated AI tab on our dashboard. Having a tool like this

has proven to improve user experience and decision-making by simplifying explanations of complex market data. We believed this addition was relevant to our algorithm trading interface.

Materials and Methods: - Connor

With our design researched and created, we were able to begin working to code our project. As referenced in the previous section, we followed our plan to develop using the basic web development languages: HTML5 for structure, CSS3 for styling, and JavaScript for any interactive behavior. These languages were foundational for the user interface, allowing the application to run without the need for excessive build tools. Along with these core three languages, we incorporated Chart.js in our Dashboard and Sector tabs for line and pie charts and Font Awesome for consistency among the icons on our site. These tools formed the basis for our frontend implementation.

A prominent change that was made this semester was the elimination of Flask and in addition, its server-side rendering. All the CSS, JavaScript, and image references are now plain relative paths, versus before, when Flask used Jinja2 template paths for static asset references. This transition has allowed the application to be packaged as a static site and deployed to an AWS S3 bucket, removing then need for a local web server.

Backend communication has also been entirely restructured in our project. Previously, the frontend had sent requests to local Flask endpoints like `/api/account` and `/api/positions` to communicate with the Alpaca API. However, in our newest version, these local endpoints do not exist anymore. Instead, the frontend communicates with an AWS API Gateway, which routes to several AWS Lambda functions that are responsible for calling the Alpaca API, processing the

responses, and returning the data to the client. Inside the JavaScript file, a shared `callLambdaAPI()` helper function is used to manage all requests, as it handles the encoded JSON body format from Lambda's proxy integration. Every API call now uses the API Gateway base URL instead of the local routes, with JavaScript now having more response parsing and error handling to accommodate the altered response format.

For organization, we structured our project by separating the HTML, CSS, JavaScript, and asset files. The main user interface is defined in a single HTML file that is stored in the root of the S3 bucket, so it can be reached directly. The application's styling is handled in a dedicated CSS file within a `css/` folder. Similarly, the API logic and interactive behavior are defined in a `script.js` file located in a `js/` folder. Our `assets/` directory contains the UConn Husky logo, which is used in the login screen and atop the sidebar, and in the new Quant AI tab. Our layout keeps every part of the interface isolated, allowing for easier debugging, updating, and extending as new features are added.

The HTML code structures the major parts of our interface and defines containers that JavaScript logic will later complete. The file is separated into two major components, the login-page container for Alpaca credential entry and the main-dashboard container that is displayed after authentication. Within the main dashboard section, the left sidebar for navigation and the main content area are both defined. All sections of the dashboard, including our newly added Quant AI, are implemented as separate `div` elements with the `tab-content` class, which JavaScript uses to show or hide sections based on if the class is active. The interactive elements of the dashboard, such as navigation buttons, trigger JavaScript functions through the inline `"onclick"` attribute. The HTML also has some placeholder containers, such as tables and metric grids, that are filled using data returned from the backend.

The CSS for the dashboard continues to be defined in a dedicated stylesheet and provides the coloring and styling for each component of the interface. We made heavy use of flexbox and CSS grid to organize our sidebar, the main content area, and card-based components. A light and dark theme are supported on our site through a defined set of CSS variables for colors, typography, and spacing. The layout is responsive as well; multiple sections such as the main graph, the sector layout, and the news feed adjust automatically based on screen size. New CSS was added this semester for our Quant AI chat interface; this included styles for the chat input area, suggestion questions, message bubbles, and the history sidebar on the left. Furthermore, we added hover and active states to certain interactive pieces like the buttons and navigation items to enhance the site's feel. The CSS allowed flexibility with the look and feel of the design, and it allowed us to create a UI theme we were satisfied with.

Housing all the functionality of the frontend is the file `script.js`, which contains the code for user authentication, tab navigation, data retrieval, and updating the interface. When a client uses their Alpaca API key and secret on the login page, `handleLogin()` catches the form submission, stores the credentials in session storage, and uses the browser's `fetch()` API to send them to the AWS API Gateway. A successful authentication hides the login container and displays the main container by toggling the CSS classes. All the following data requests use the same API Gateway base URL.

The tabs in the dashboard are controlled using the `switchTab()` function, which toggles the active class to show or hide the current tab. Many of our interface sections start as empty placeholder HTML containers that are populated dynamically once the API Gateway can retrieve live data. `Chart.js` is used to create charts by instantiating new Chart objects tied to canvas elements in the HTML, which later get updated with data and labels. The Performance Overview panel was

updated this semester to compute real figures from the Alpaca portfolio history data, since it previously was inaccurate.

The new Quant AI tab is powered by a Lambda endpoint on the same API Gateway used for getting /positions and /account. The JavaScript is responsible for handling multiple chat sessions, with each getting stored as an array of message objects with role and content fields. When a chat is active and a message is sent, the entire history of the conversation is given to the Lambda endpoint, and in response, an AI-generated message is returned in the chat area in a message bubble. This function allows users to chat with AI and get answers to any questions on momentum trading, market conditions, and risk management of their portfolio.

The company logo display logic was readjusted and fixed this semester. The previous logic used was an API call from the frontend to Clearbit API to use a company domain lookup. However, this logic was failing to load logos more than it was succeeding. To fix this, an updated system was created using the Parquet API. First, the stock logos from Parquet are tried directly, and if this fails, then Clearbit is tried next. If these first two attempts fail, then the first two characters of the company's ticker symbol are displayed where the logo should go. This change was a significant improvement, as almost every company in our holdings now has a visible logo.

Our settings panel was completely reworked to simplify the options as much as possible. We previously had unclear directions on how to receive notifications, including a useless phone number input and an instruction block on how users could run a PowerShell script to complete an email subscription. We have completely removed all the vague parts of the notification functionality, and the email subscription feature is now handled completely through the backend on AWS. In this panel, we have kept the trading parameters, risk management controls, and the

email subscription field. We also newly added inline tooltips to the settings, which display explanations on all the setting fields to contextualize complex financial parameters for users.

The final area that was reoriented was the sector classification system. The way the sectors were previously organized was heavily reliant on the Alpaca API, which raised API rate limit problems and some misclassifications. The updated methodology takes a large, hardcoded GICS-aligned sector map with a majority of the S&P 500 companies and uses a `groupBySector()` function to classify our positions without any API calls. If a position isn't in this hardcoded list, it will still fall back to the Alpaca API to get a classification, but this list has reduced API calls and increased the speed and accuracy of the groupings significantly.

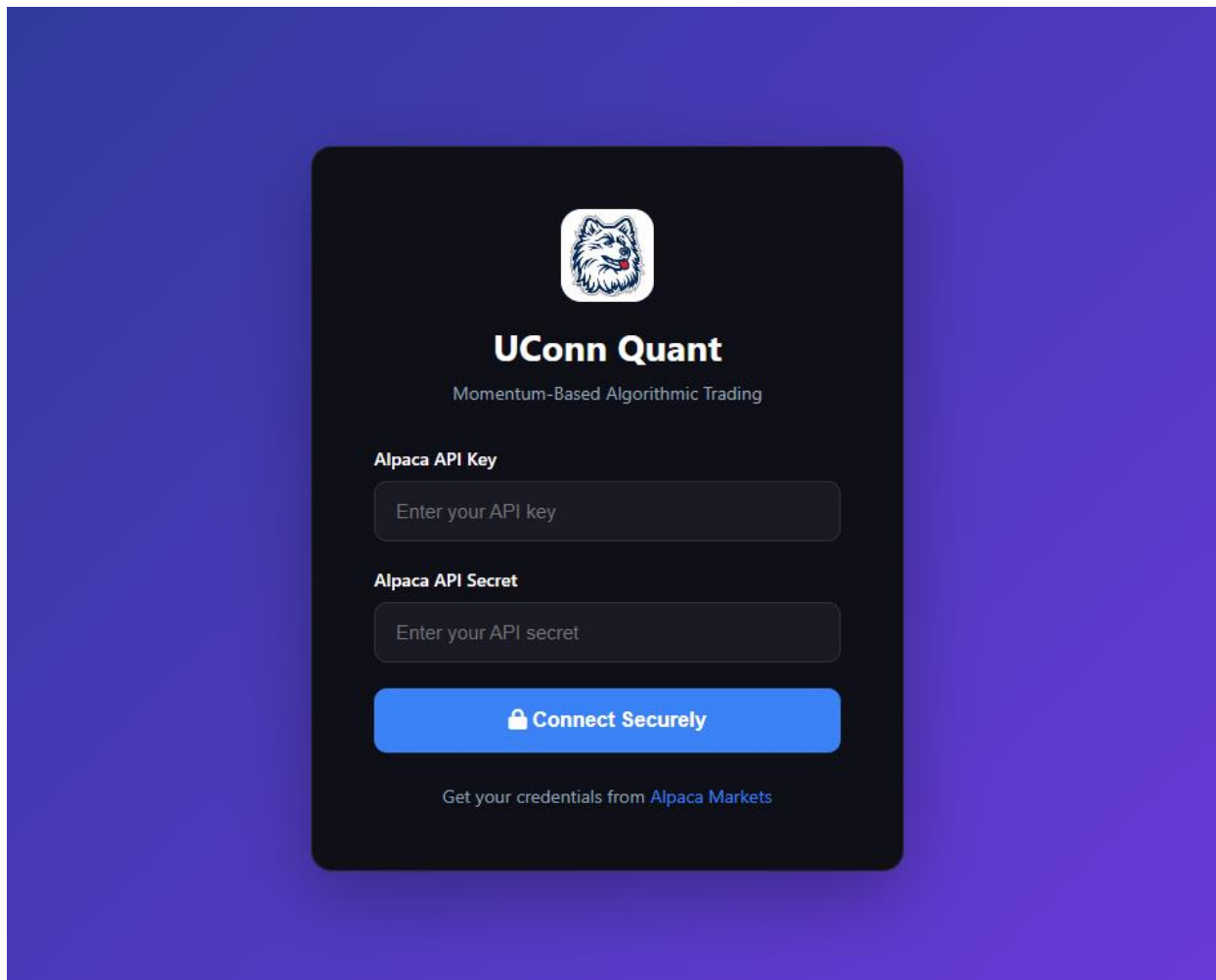
Altogether, JavaScript provides the function control for the interactivity of the dashboard and allows the interface to update in real time based on the data returned from the AWS API Gateway. The shift from a dependency on Flask to an AWS hosted static S3 site with communication through API Gateway and Lambda has made the system more sustainable and production-ready than previously.

Preliminary Results: - Peter

The current frontend implementation of our *UConn Quant* platform provides a fully functional and interactive interface which allows users to use their Alpaca trading accounts to visualize live portfolio performance in real time. Upon accessing the website, users are prompted to authenticate using their Alpaca API credentials, once logged in, the dashboard dynamically loads account metrics, open positions, sector allocations, and historical performance data through responsive charts and tables. Our interface provides navigation tabs which include the Dashboard, Positions, Sectors, News Feed, Quant AI, and Settings, each of which presents

different aspects of the trading system's data. Within these tabs, the user can interact with the different timeframes, toggle light/dark mode, real-time news information, and expandable sector views.

When the app is started, the very first thing users see is a special login screen made to get their Alpaca API Key and Secret. The interface is in the middle of the page, has the UConn Husky logo, and is decorated with a gradient background to give off a professional vibe right from the start. Users enter their Alpaca configurations, and select "Connect Securely", and are immediately transitioned to the main dashboard if the information is correct.



Once authenticated, the system stores the credentials temporarily in session storage, allowing the

user to navigate the dashboard without repeated prompts. This login mechanism enables secure, session-based access to real-time account information pulled directly from Alpaca's API, ensuring that all displayed performance data, positions, and news are fetched specifically for the authenticated user.

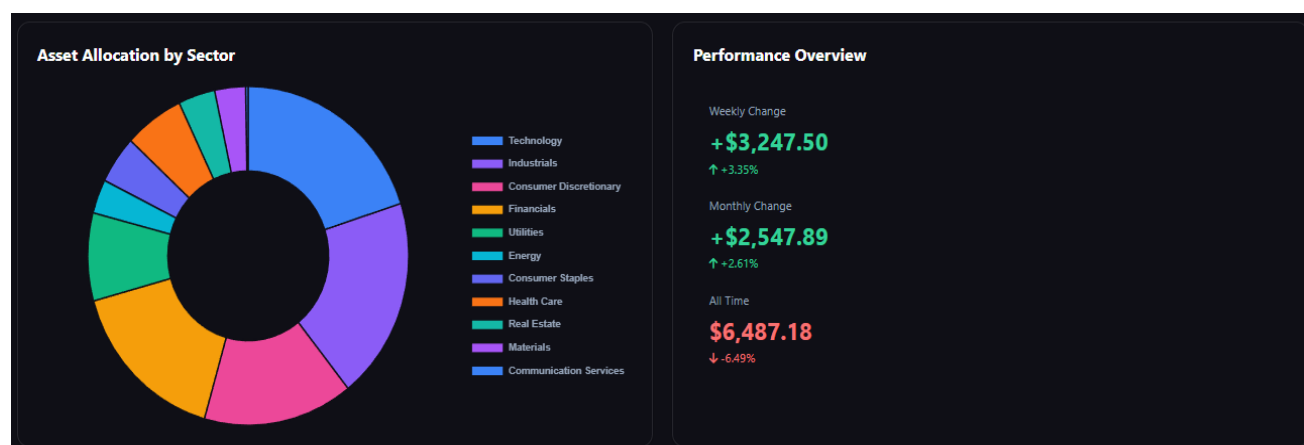
When the user successfully logs in, they are automatically shown on the Dashboard tab, with a set of dynamic performance metrics that summarize the state of the portfolio in real time. These metrics include the following: Total Value of Fund, Total Profit, Daily Change, Best Performer, Worst Performer. Each of these metrics is calculated with the most recent Alpaca accounts position data. These metrics are color-coded to show positive or negative performance, and they are automatically updated when new data is fetched. These cards provide users with a quick, high-level understanding of the portfolio's health and help users identify their strongest and weakest contributors to their overall performance.



As seen in the above image, directly below the metrics, we provide the Active Performance Line Chart, which visualizes the historical performance of the user's portfolio. This chart dynamically loads the data from the Alpaca's `/account/portfolio/history` endpoint. This endpoint supports multiple time periods, including 1D, 1M, 3M, YTD, 5Y, and Inception-to-Date (ITD). Switching between these ranges will immediately update the chart with newly fetched or processed data, allowing the users to examine their short-term and long-term growth trends. Hovering the mouse over specific points on the graphs will show a small pop up of the portfolio value at that moment, allowing for a more precise and interactive examination of performance changes over time.

The dashboard also includes a Sector Allocation Pie that provides a visual breakdown of how the portfolio is distributed amongst the different market sectors. These sectors include Technology, Financials, Industrials, Consumer Discretionary, and others. The chart calculates the percentage

of total portfolio value represented by each sector, which is displayed using a multi-color chart for readability and differentiation.

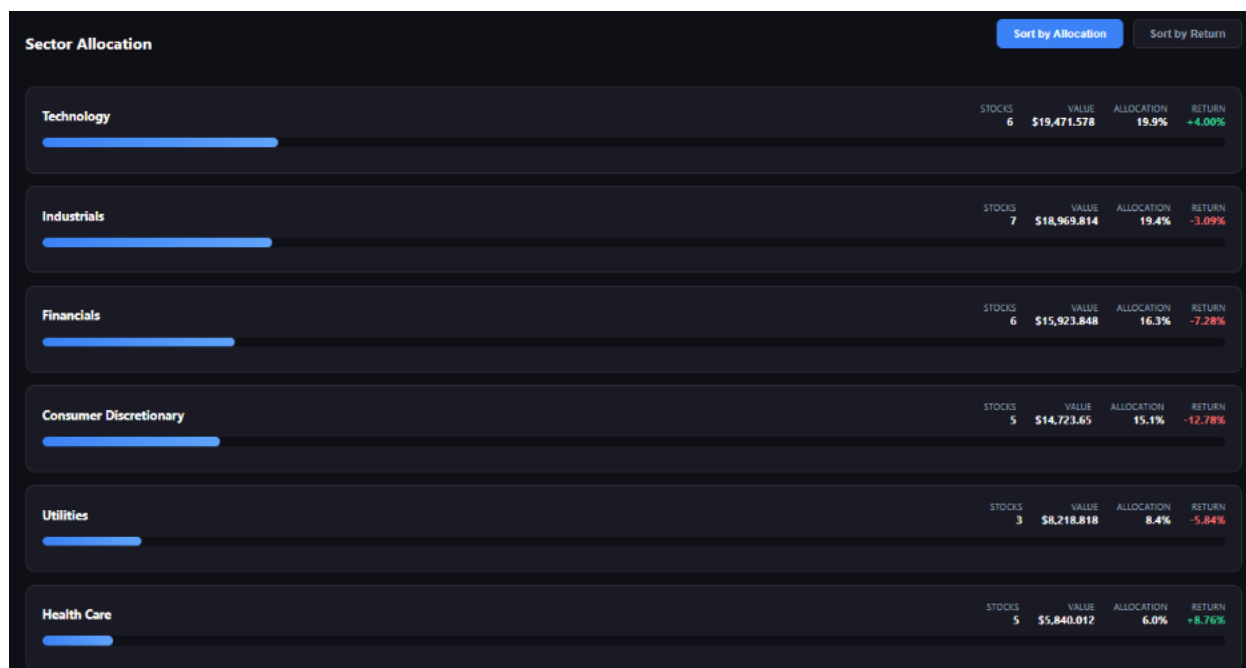


The chart also updates dynamically when new account data is fetched, and hovering over any slice reveals the exact allocation percentage. To the right of the Sector Allocation Pie Chart, the dashboard presents a Performance Overview panel summarizing short-term and long-term changes in portfolio value. This panel includes Weekly Change, Monthly Change, and All-Time Performance, each displayed with both dollar and percentage movement. Once again, the positive values are represented in green with an upward arrow, while the negative values are represented in red with a downward arrow.

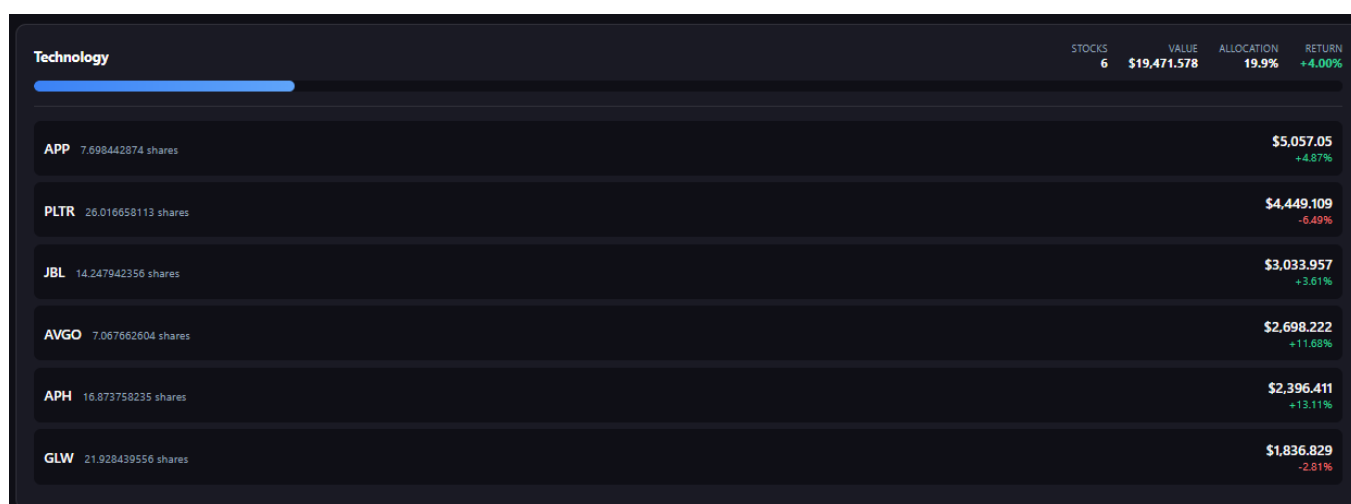
The next tab the user has access to is the Positions tab. This shows a detailed breakdown of all the assets currently held within the user's portfolio. These holdings are automatically sorted in descending order based on their market value, ensuring that the most significant positions are displayed first. Additional details include the number of shares held, current price, total market value, unrealized profit or loss, and percentage return.

Positions							Logout
All Holdings							
COMPANY		SHARES	PRICE	MARKET VALUE	GAIN/LOSS	RETURN %	
 GLD GLD		10.899	\$438.65	\$4,780.813	\$-219.18	-4.38%	
 HOLX HOLX		47.673	\$76.01	\$3,623.615	+\$37.18	+1.04%	
 EA EA		11.555	\$203.84	\$2,355.344	+\$48.18	+2.09%	
 ROST ROST		7.777	\$225.47	\$1,753.569	+\$201.28	+12.97%	
 MNST MNST		19.856	\$75.31	\$1,495.368	\$-133.63	-8.20%	
 GOOGL GOOGL		4.418	\$314.10	\$1,387.795	+\$43.56	+3.24%	
 CTVA CTVA		16.417	\$84.43	\$1,386.104	+\$128.05	+10.18%	

The Sectors tab provides a more detailed view of the portfolio composition by breaking all holdings into their respective market sectors and displaying performance information for each group. When this tab loads, the page shows key metrics such as the sector value, allocation percentage, average sector return, and number of holdings. Once again, we wanted to follow the same color coding where positive values are represented in green with an upward arrow, while the negative values are represented in red with a downward arrow. Users can sort sectors either by allocation percentage or average return, allowing them to quickly identify which areas of the portfolio are driving performance.

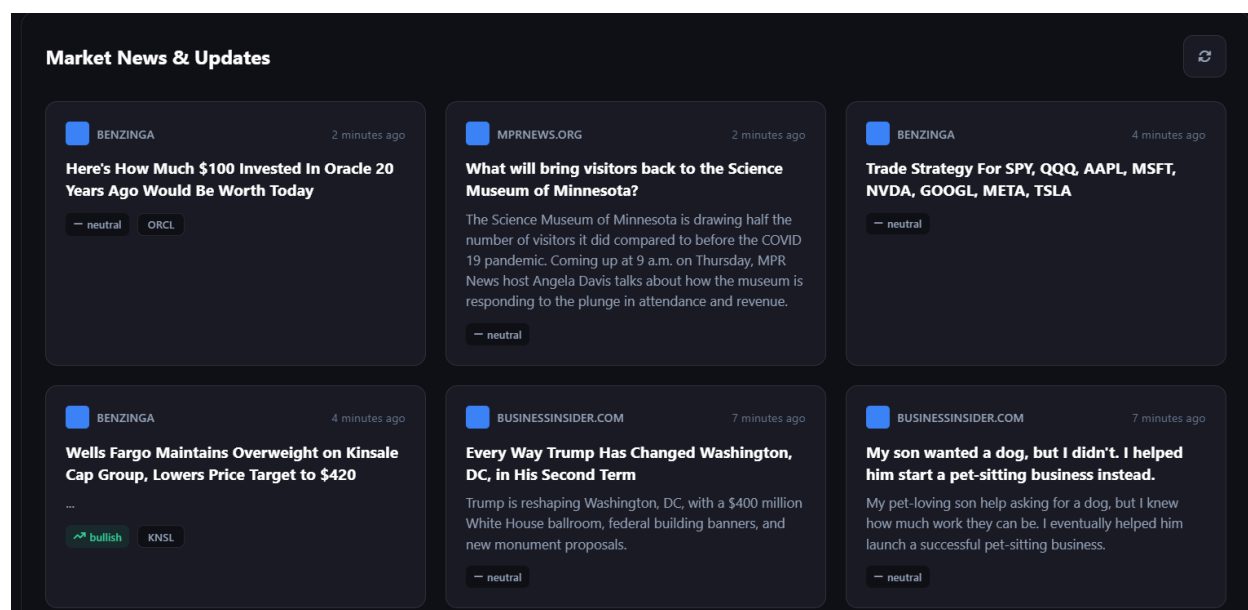


Each sector is displayed as an interactive card; each card is expandable when clicked. The dropdown will reveal the companies along with their number of shares, market value, and stock level return percentages. All values update dynamically whenever new position data is fetched, ensuring accurate sector representation even as market conditions change.



The News Feed tab provides users with a stream of the latest market headlines directly sourced from Alpaca News (Bezinga), Finnhub, and MarketAux public API's. The frontend sends live

requests to retrieve up to 20 of the most recent articles, each of which is displayed as its own card with the article source, publication time, headline and a short summary. Each card contains a sentiment level, which is a quick indicator of whether news suggests the market is likely to move up (bullish), down (bearish), or neither (neutral). Following the principle of green indicating upward/positive movement and red downward/negative movement, we have color-coded these sentiment levels as well.

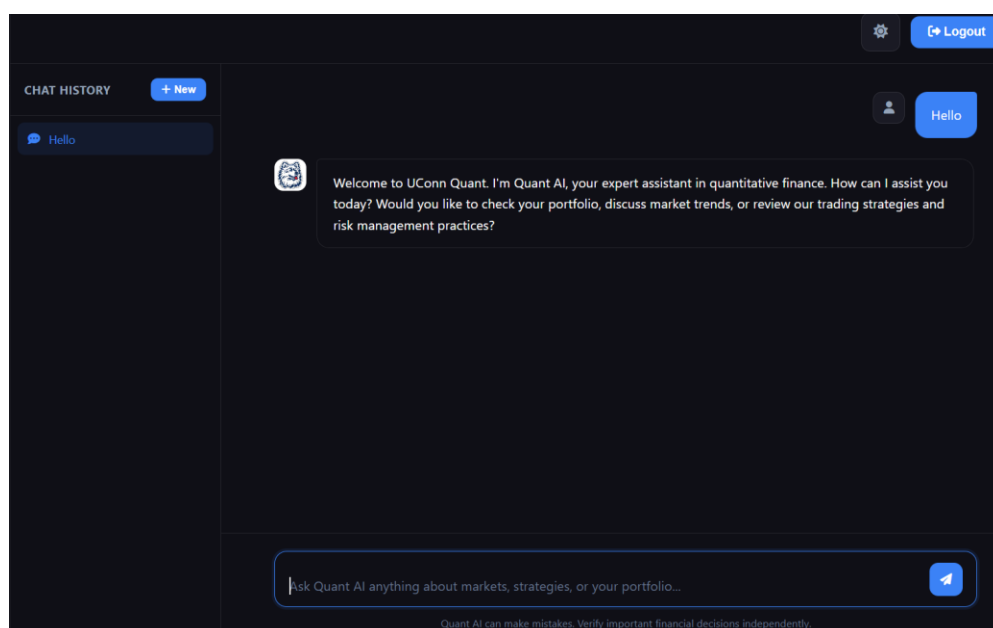


If Alpaca's API is unavailable or returns no articles, the system falls back on a predefined set of sample news items to maintain the layout and functionality of the frontend. Each card in the feed is fully interactive, opening the full article in a new tab when clicked. The feed supports dynamic refresh via a dedicated reload button, allowing users to pull in new headlines at any time without leaving the dashboard. Time stamps are provided to show how recently these articles were published.

The Quant AI tab gives users access to an AI assistant built directly into the platform to help answer finance and portfolio-related questions. Instead of acting like a general chatbot, this

assistant is designed to respond like a quant-focused financial helper, meaning it is meant to answer questions about market trends, trading strategies, portfolio performance, sector movement, and general investing ideas. Users can type in their own questions or choose from example prompts already shown on the screen, making it easy to interact with and useful for both learning and quick financial insight.

To make its responses more useful, the Quant AI is powered by Groq AI and supported by Tavily news sources, which helps it include recent market information and current financial events in its answers. This allows the assistant to go beyond simply explaining general investing topics by also helping users better understand what may be happening in the market at the moment. Overall, this feature adds another layer to the platform by giving users a more interactive way to explore their portfolio, ask questions, and get quick financial guidance all in one place.



Conclusions: - Peter

The UConn Quant platform's frontend is able to produce an interactive dashboard for users that displays live portfolio performance data, market news, sector distributions, and user-configurable options. Through integration with the Alpaca API, the platform is capable of retrieving account information as well as orders placed on behalf of a user. The dashboard provides a user-friendly interface that presents information in an organized and visually accessible manner. It is able to visualize a user's equity curve, dynamically update chart data based on selected timeframes, display sortable portfolio and sector tables, and provide customizable alerts for different trading-related conditions. The frontend has demonstrated many of the core capabilities expected from a modern algorithmic trading visualization platform.

In addition to the frontend dashboard, the platform was further expanded through the successful implementation of an integrated Quant AI assistant, which is deployed as part of the website hosted through an AWS static S3 bucket architecture. This feature allows users to interact with an AI assistant designed specifically for financial and market-related questions, adding a more intelligent and interactive layer to the overall user experience. The assistant was configured to behave as a quant-focused financial tool and is supported by Groq AI along with Tavily-based news sourcing, allowing it to provide more useful and market-aware responses.

The platform was also improved through the completion of the Email Notification System, which is now automated through AWS-based cloud services rather than relying on a manual external script. This enhancement allows the system to better support automated delivery of important user notifications such as market-open summaries, daily performance summaries, and trade alerts. By connecting the notification system to a more persistent cloud-based backend process, the platform now offers a more reliable and scalable alerting workflow.

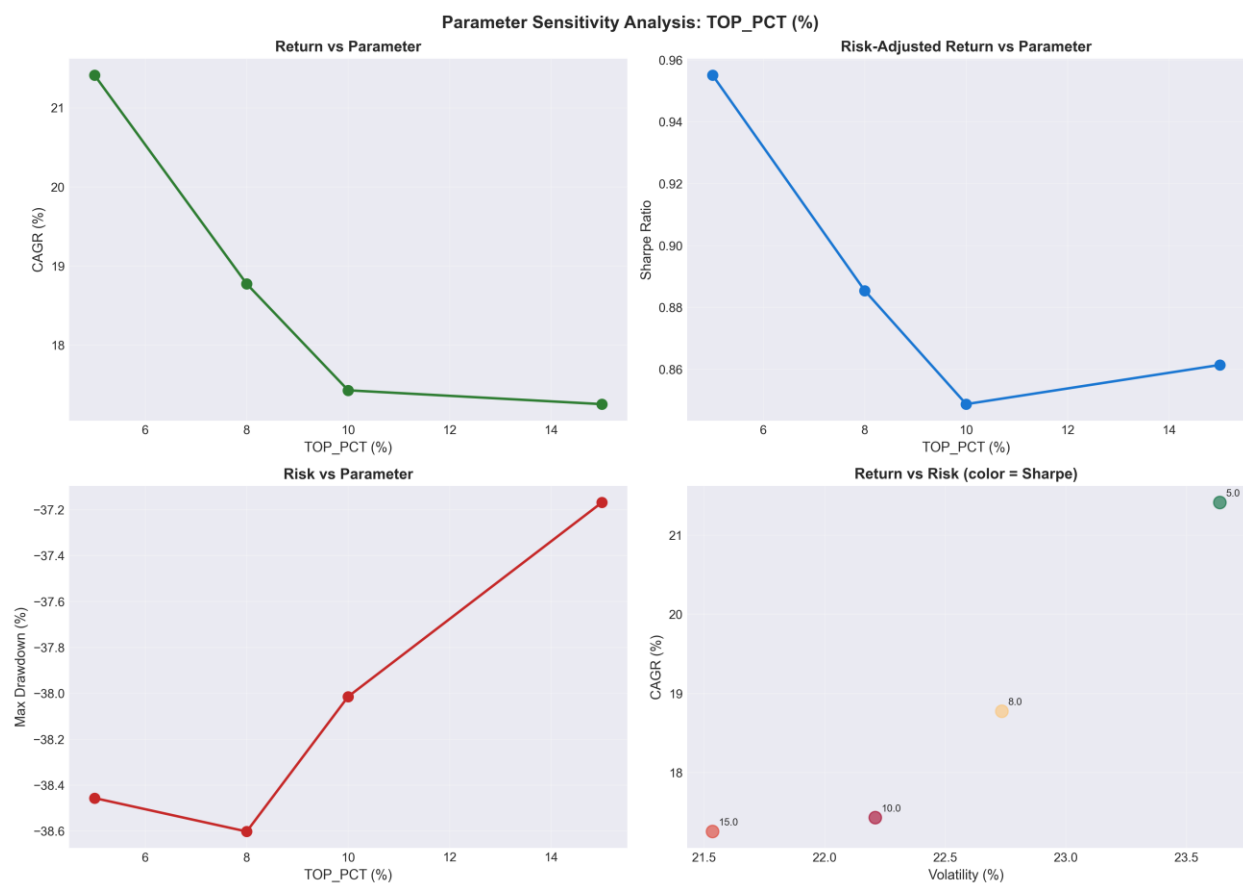
Another important improvement made during development was the expansion of the News Feed system through the addition of more free and accessible news sources. This helps reduce dependence on limited or paywalled content and improves the quality, variety, and usability of market information available to users directly within the dashboard. As a result, users are able to access a broader stream of financial news without leaving the platform.

Overall, the completed UConn Quant platform represents a fully developed and highly functional trading dashboard that combines live portfolio tracking, sector and performance analytics, financial news integration, automated cloud-supported notifications, and AI-assisted market interaction into a single unified system. These completed features make the platform not only informative and visually interactive, but also more practical, intelligent, and scalable for real-world portfolio monitoring and decision support.

Factor Considerations – Connor

We considered several factors in the development of our trading algorithm and over the course of the project's life cycle. Public health and safety are not typically factors that a financial algorithm would influence but features in our algorithm including volatility targeting and drawdown controls ensure financial stability and safety for users. Through the development of our systematic strategies balancing both risk and return, we are directly supporting investor confidence and the preservation of wealth. Globally speaking, our focus on the S&P 500, which operates in an international marketplace, is influenced by geopolitical events around the world and market performance can be affected as a result. Socially and culturally, our methods are reflective of modern financial practices and have an emphasis on rule-based decision-making, which encourages market fairness. Our strategy is directly shaped by economic cycles and macroeconomic conditions, and we test these across different regimes from 2010 to just recently.

Environmentally, our algorithm is not necessarily applicable, but we recognize the expanding importance of environmental considerations in investments.



Works Cited

Asness, C. S., Moskowitz, T. J., & Pedersen, L. H. (2013). *Value and Momentum Everywhere*. The Journal of Finance, 68(3), 929–985.

Carhart, M. M. (1997). *On Persistence in Mutual Fund Performance*. The Journal of Finance, 52(1), 57–82.

Jegadeesh, N., & Titman, S. (1993). *Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency*. The Journal of Finance, 48(1), 65–91.

Jegadeesh, N., & Titman, S. (2001). *Profitability of Momentum Strategies: An Evaluation of Alternative Explanations*. The Journal of Finance, 56(2), 699–720.

Nielsen, J. (2024). *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>